Kathmandu urban heat: what drives hot neighbourhoods, and who is exposed?

Project overview and architecture · Abiral Gautam · draft of 17 Oct 2026. No model results yet: every number that comes from code will be added after it is run.

The question

Within Kathmandu valley, which neighbourhoods run hottest in the pre-monsoon season, what explains it (green cover, built-up land, elevation, distance to water, roads), and where do hot areas overlap with dense population?

What the project does, in plain words

It takes satellite pictures of Kathmandu and turns the valley into about 1,000 hexagon-shaped neighbourhoods of roughly 0.7 km^2 each. For every hexagon it measures the average temperature, how green it is, its height, its land cover and how many people live there. A Random Forest model learns which of these features go with hot hexagons. We test it honestly by hiding whole areas from it. Finally, hot areas with many people are flagged on an interactive map as the most exposed.

- It does **not** forecast future temperatures.
- It does **not** prove that buildings or missing trees *cause* heat. It shows patterns.
- It measures daytime *surface* temperature from space, not what a person feels.

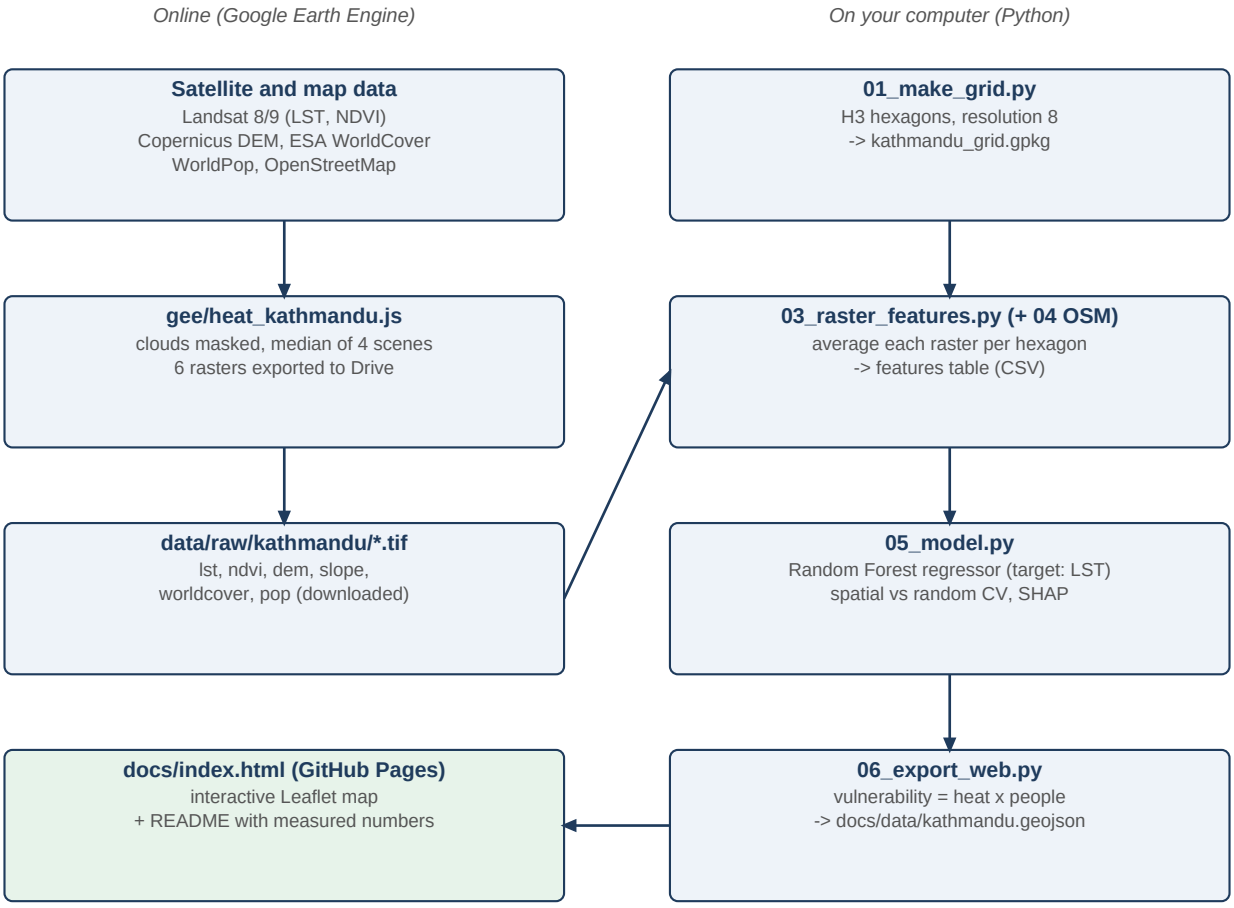
Why it fits the lab

It is a climate extreme in an urban setting, analysed with satellite data and machine learning at neighbourhood scale, with a vulnerability layer that asks who is exposed. That matches the Urban Analytics Lab's focus on urban resilience and GeoAI, and it connects to the climate-justice theme of the lab paper “Sensing climate justice”.

Pipeline and status

Step	What happens	Status
1. Grid	H3 hexagons, resolution 8, about 1,056 cells over the study box	Done
2. Satellite data	Landsat 8/9 LST and NDVI, plus elevation, slope, land cover, population	Script run; exports to start
3. Features	Average each raster inside every hexagon (zonal statistics)	Code to adapt
4. OSM (optional)	Road density and distance to the nearest waterway	Written, untested on real data
5. Model	Random Forest regressor on mean LST; spatial vs random CV; SHAP	To adapt
6. Vulnerability and map	Heat x normalised log population density; Leaflet map on GitHub Pages	To adapt

Architecture



Left column: data is prepared online in Earth Engine and downloaded. Right column: Python scripts build the grid, attach features, train and test the model, compute the vulnerability index and write the file the web map reads. Each script reads the output of the one before it, so any stage can be re-run on its own.

Data sources

Layer	Dataset	Native resolution	Used for
Temperature (LST)	Landsat 8/9 Collection 2 Level 2, band ST_B10	about 100 m	Model target
Greenness (NDVI)	Landsat 8/9 surface reflectance	30 m	Feature
Elevation, slope	Copernicus DEM GLO30	30 m	Features
Land cover	ESA WorldCover v200 (2021)	10 m (exported at 30 m)	Feature (share of each class)
Population	WorldPop 100 m, Nepal, 2020	100 m	Vulnerability index only
Roads, waterways	OpenStreetMap via osmnx (optional)	vector	Features

Model design

- **Target:** mean LST of each hexagon, in °C. **Features:** NDVI, elevation, slope, land-cover shares, and (optional) road density and distance to waterway.
- **Left out of the features:** LST itself and population. Population is kept for the vulnerability index so the model cannot lean on it.
- **Model:** Random Forest regressor (scikit-learn). Settings are fixed in advance and several random seeds are reported.

- **Random CV vs spatial CV:** random folds mix neighbouring hexagons across train and test, which flatters the score. Spatial folds keep whole blocks (H3 resolution 6 parents) together, so the test areas are truly unseen. The gap between the two is reported as a finding.
- **Metrics:** R^2 and RMSE per fold. **Explanation:** SHAP values show what the model used, not what causes heat.
- **Vulnerability index:** predicted or measured heat, scaled 0 to 1, x normalised log population density.

Repository layout (planned)

```
kathmandu-urban-heat-geoai/
  config.yaml             city, bbox [west,south,east,north], UTM 32645, H3 res 8, CV settings
  gee/heat_kathmandu.js   Earth Engine script (LST, NDVI, DEM, slope, land cover, population)
  src/01_make_grid.py     hexagon grid
  src/03_raster_features.py   rasters -> one row per hexagon
  src/04_osm_features.py  optional road and waterway features
  src/05_model.py         Random Forest, random vs spatial CV, SHAP
  src/06_export_web.py    vulnerability index and GeoJSON for the map
  data/raw/kathmandu/     the 6 downloaded .tif files (not committed if large)
  data/processed/         grid and feature tables
  outputs/                metrics, predictions, SHAP plot
  docs/                   index.html and data/kathmandu.geojson (GitHub Pages)
  README.md               method, results (only numbers the code produced), limits
```

Data checks so far

- Landsat 8/9 scenes with under 30% cloud, 15 Mar to 31 May 2024: **4** (14 Apr, 30 Apr, 16 May, 24 May).
- Earth Engine composite (median of the 4 scenes): LST min 19.4, mean 35.0, max 49.2 °C; valid share of the box 1.0. The 49 °C maximum still needs a check on the map (expected: bare ground, dark roofs, dense built-up land).
- A first attempt at a flood-extent project was dropped: Sentinel-1 had no usable image at the September 2024 flood peak, so there were too few flood labels. This goes in the README as a short note.

Honest limits (for the README)

- Four scenes, one season, one year. The thermal band is about 100 m natively and the scenes are about 10:30 AM only.
- The model describes patterns in this valley; it is not a forecast and not a causal claim.
- The LST and population overlap measures exposure, not harm. WorldPop is a modelled estimate.
- Neighbouring hexagons are similar, so spatial CV is used and its gap to random CV is reported.

Timeline

- **Sat night:** run the 6 Earth Engine exports, download the .tif files, check them.
- **Sun morning:** stages 3 to 5 (features, model). Hard stop for modelling around 1 PM.
- **Sun afternoon:** web map and README.
- **Sun evening:** cover letter and documents (CV, signed NUS consent form).
- **Mon morning:** final check and submit before noon (Nepal time). The deadline is 20 Oct.